3.6. Современные технологии разработки программного обеспечения

3. Сети, базы данных, безопасность кода

В этом уроке

- Работа с репозиториями, команды init, add, commit, push, совместная разработка.
- Контейнеры, образы, изоляция сред, пример запуска Python-приложения.
- Анализ вредоносного кода в безопасной среде, тестирование уязвимостей.
- Создание Docker-образа и запуск контейнера с простым Python-скриптом.

Практическое задание

Создайте папку my_project и инициализируйте Git-репозиторий.

Добавьте файл script.py с кодом print("Hello, Git!").

Зафиксируйте изменения с сообщением "Добавлен скрипт".

Измените файл, добавив строку print("New feature"), и зафиксируйте изменения.

Просмотрите историю коммитов.

Практическое задание

В репозитории из задания 1 создайте ветку feature.

Добавьте в script.py функцию def test(): print("Test") и зафиксируйте изменения.

Переключитесь на ветку main и слейте изменения из feature.

Удалите ветку feature.

Практическое задание

Установите Docker (если не установлен).

Запустите контейнер с Python в интерактивном режиме.

Напишите код print("Hello, Docker!") прямо в контейнере.

Создайте файл Dockerfile для образа с Python и скриптом app.py

Практическое задание

Создайте файл app.py с кодом print("This is my app!").

Напишите Dockerfile для этого приложения.

Соберите образ и запустите контейнер.

Практическое задание

Создайте репозиторий с файлом malware_analyzer.py (пустой скрипт).

Зафиксируйте изменения в Git.

Создайте Dockerfile для запуска скрипта в контейнере.

Запустите контейнер, чтобы проверить изоляцию.

Скринкаст

Итоги урока

- Освоили базовые команды Git для управления версиями кода.
- Научились создавать Docker-образы и запускать контейнеры.
- Поняли, как использовать контейнеры для изолированного тестирования и анализа безопасности.
- Применили навыки для автоматизации разработки и обеспечения безопасности в реальных задачах.